Killing bloat

Retour à « Resources »

Killing Bloat

Understanding your agent's payload

Every request you send to your agent carries an invisible payload: tool schemas, the system prompt, the skills catalogue, and feature instructions all ship together — and you are billed for every token, on every turn. Before you type a single word, the bill has already started.

The problem is visibility. `/context` breaks the window into categories, but lumps every tool into one number. To cut precisely, you need to see the actual payload.

See what actually gets sent

Put a logging proxy between your agent and the API. `agent-proxy` is a small one: it writes a Markdown report of exactly what was sent to the model on every turn — the system prompt, every tool schema, the whole message history — ranked by size.

```
# in one terminal, following the gist's setup
node agent-proxy.mjs

# in another, in your repository
ANTHROPIC_BASE_URL=http://localhost:8787 claude
```

Then: start a **fresh** session, send your agent a single `Hello!`, wait for the reply, and open the report the proxy just wrote. One turn, so one report, and nothing else in it to read past.

Read the report

That one "Hello!" typically renders to 4,000+ lines and 150–200 KB. All of it is sent and billed every turn. Start with the ranking — it tells you what to cut before you have read a word — then go through it:

- `<system-prompt>` — note the **Environment** section (OS, shell, working directory), the **Context management** section, and, on agents like Claude Code, your recent git commits.
- `<tools>` — count the definitions (likely 70+). Some you use daily; many you have never heard of — `CronCreate`, `DesignSync`, `EnterPlanMode`, `Workflow`. Notice how verbose some are, with whole paragraphs on commit message formats, PR templates, and feature flags.
- **`mcp__`** — MCP tools attached via integrations (`mcp__claude_ai_Figma__authenticate`, `mcp__claude_ai_Gmail__create_draft`, `mcp__claude_ai_Slack__authenticate`, `mcp__claude_ai_Google_Drive__search_files`, …). Count them. They ship whether you need them or not.
- **"following skills are available"** — the skills catalogue, usually 15–20+ entries. Note which are yours and which came bundled with the agent.

Finally, run `/context` in your agent and write down the number. That is your baseline, and the rest of this document is about bringing it down.

The first step is to create a `settings.json` file inside your `~/.claude` directory and initialize it with an empty object.

The instructions demonstrated here are agent-specific, but if you look at the written lesson below, you'll see harness instructions for removing bloat there too if needed.

The workflow

Every time you make a change to `settings.json`, the process is the same:

1. Edit `~/.claude/settings.json`
2. Quit your agent
3. Relaunch it
4. Send a `Hello!` message
5. Run the `/context` command
6. Open the new report to see what's changed

To give us a baseline marker, let's run `/context` on a fresh start. We're starting at **68.3k tokens**. That's really quite a lot.

Disable MCP connectors

The first thing to address is the MCP (Model Context Protocol) stuff. There's a bunch of Claude AI connectors that automatically enable if you're using Claude.ai.

Inside `~/.claude/settings.json`, you can disable all of them in one go:

```
{
  "disableClaudeAiConnectors": true
}
```

After restarting and running `/context`, we've already dropped down to **47k tokens**. That's 21k tokens saved with a single setting.

Disable workflows

Next, you'll notice there's a ton of configuration related to dynamic workflows. These orchestrate multiple sub-agents deterministically.

If you won't be using workflows, disable them:

```
{
  "disableClaudeAiConnectors": true,
  "disableWorkflows": true
}
```

Now we're down to **39k tokens**.

Disable bundled skills

The bundled skills include deep research, data visualization, artifact design, config updates, and keybindings help. If you're not using any of them, disable them all:

```
{
  "disableClaudeAiConnectors": true,
  "disableWorkflows": true,
  "disableBundledSkills": true
}
```

This takes us down to **37.1k tokens** — and kills skills usage completely.

Disable artifacts

There's a relatively new feature called Artifacts that you might not use at all. Turning this off is straightforward:

```
{
  "disableClaudeAiConnectors": true,
  "disableWorkflows": true,
  "disableBundledSkills": true,
  "disableArtifact": true
}
```

That's another 4k tokens gone, bringing us to **33.1k tokens**.

Control tool access with permissions

Now for the big one: looking at the tool definitions themselves.

Here's the key principle: if you deny your agent access to a tool via `permissions.deny`, its definition is removed from the system prompt entirely. This isn't just a runtime block — it's a context saving on every single request.

Let's look at which tools you actually need:

Tool	Ask yourself…	Action
`DesignSync`	Do you need to design, sync, read, and update Claude.ai design-system projects?	No. Deny it.
`CronCreate`, `CronDelete`, `CronList`	Do you need to create and manage cron jobs?	Probably not. Deny all three.
`EnterPlanMode`, `ExitPlanMode`	Do you need the agent to enter and exit plan mode?	No. Deny both.
`PushNotification`, `RemoteTrigger`	Do you need push notifications or remote triggers?	No. Deny both.
`ReportFindings`	Do you need the agent to report findings?	No. Deny it.
`ScheduleWakeup`	Do you need wake-up scheduling?	No. Deny it.

Add these to your settings:

```
{
  "permissions": {
    "deny": [
      "DesignSync",
      "CronCreate",
      "CronDelete",
      "CronList",
      "EnterPlanMode",
      "ExitPlanMode",
      "PushNotification",
      "RemoteTrigger",
      "ReportFindings",
      "ScheduleWakeup"
    ]
  },
  "disableClaudeAiConnectors": true,
  "disableWorkflows": true,
  "disableBundledSkills": true,
  "disableArtifact": true
}
```

This takes us down to around **21.6k tokens**.

One more tool to consider

There's one more tool worth disabling: `AskUserQuestion`. It's a very long tool definition (~130 lines), and some people love the UI while others find it intrusive.

```
{
  "permissions": {
    "deny": [
      "DesignSync",
      "CronCreate",
      "CronDelete",
      "CronList",
      "EnterPlanMode",
      "ExitPlanMode",
      "PushNotification",
      "RemoteTrigger",
      "ReportFindings",
      "ScheduleWakeup",
      "AskUserQuestion"
    ]
  },
  "disableClaudeAiConnectors": true,
  "disableWorkflows": true,
  "disableBundledSkills": true,
  "disableArtifact": true
}
```

Removing it takes us down to **under 20k tokens** — around 19.9k. That's a good stopping point.

The attitude matters more than the specifics

This demonstration is agent-specific, but the attitude is what matters.

You have to be aware of all the tokens you're sending in your system prompt. You don't need to read every single one, but you need to notice if something's there that shouldn't be — because it affects every subsequent request.

Performing this kind of hygiene on your own tools and setup is really important. Notice that everything we removed here was stuff you weren't going to use anyway. It absolutely shouldn't be in there, influencing behavior and eating up precious tokens.

Your turn

Go and look at your own harness. See what you can remove from the system prompt while still retaining good behavior. Find your own harness-specific optimizations, and when you do, share them with the class.

Now you have a good base to start working from. Let's get building.

Credits

aihero.dev (Matt Pocock).